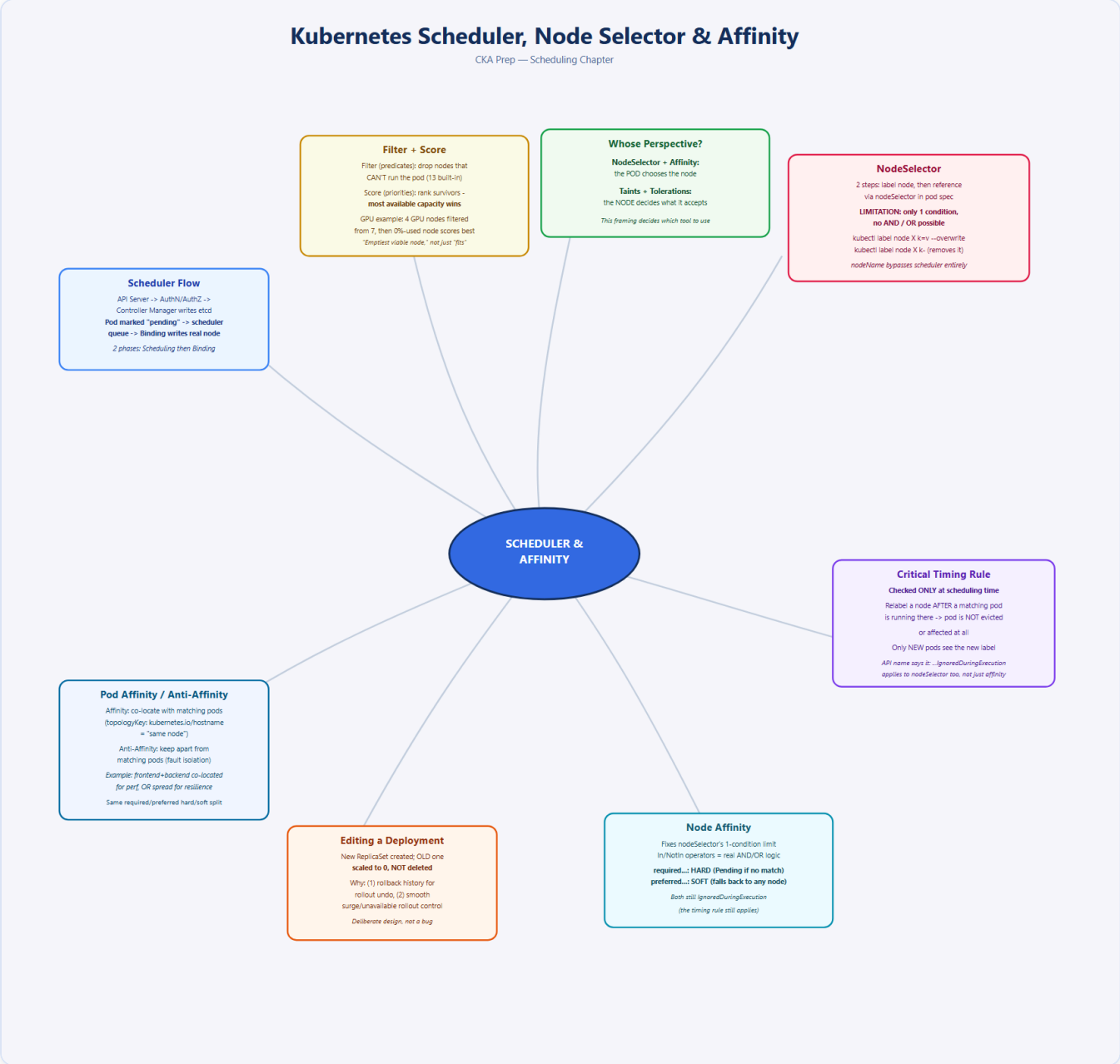
KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 06

Kubernetes Scheduler, Node Selector & Affinity

Topic: Scheduling Internals | Core Kubernetes (not ArgoCD) | 2026-09-14

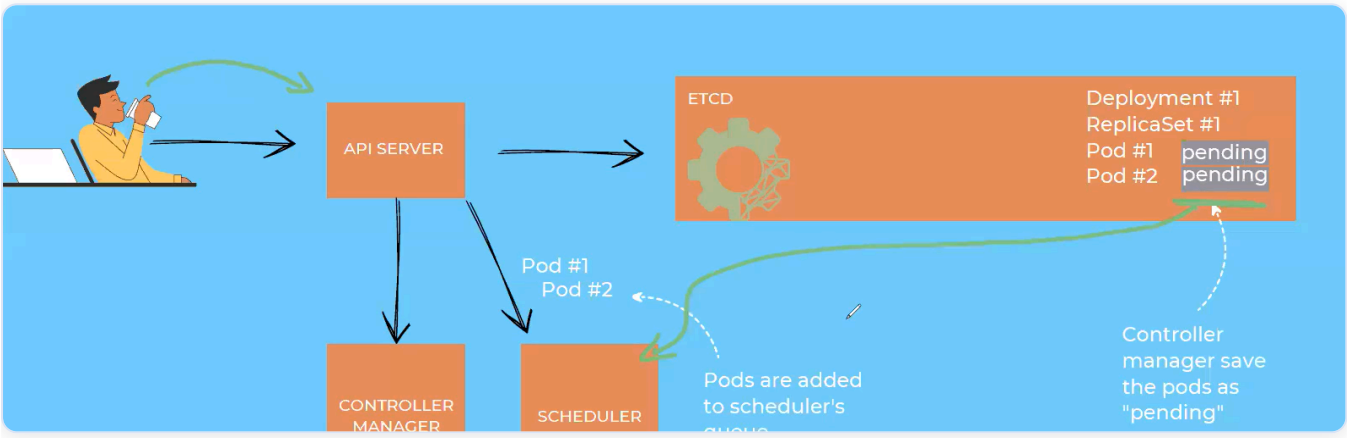
Quick Recall — Mind Map



The 3 Ways to Influence Scheduling

In Kubernetes, there are three mechanisms to influence where pods land: **Node Selector**; **(Node) Affinity/Anti-Affinity and Pod Affinity/Anti-Affinity**; **Taints and Tolerations**. Before learning how to *influence* scheduling, first understand how scheduling actually works without any influence at all.

How the Scheduler Works, End to End



Step 1 — Request Lands, Gets Stored as "Pending"

- User runs `kubectl create -f deploy-abc.yaml`.
- Request hits the **API Server** — checks **AuthN/AuthZ** first; if valid, proceeds.
- The **Controller Manager** creates Deployment → ReplicaSet → Pod entries in **ETCD**.
- Pods are marked **"pending"** in ETCD — they exist as objects, not assigned to any node yet.
- Any pending pod is added to the **scheduler's queue**.

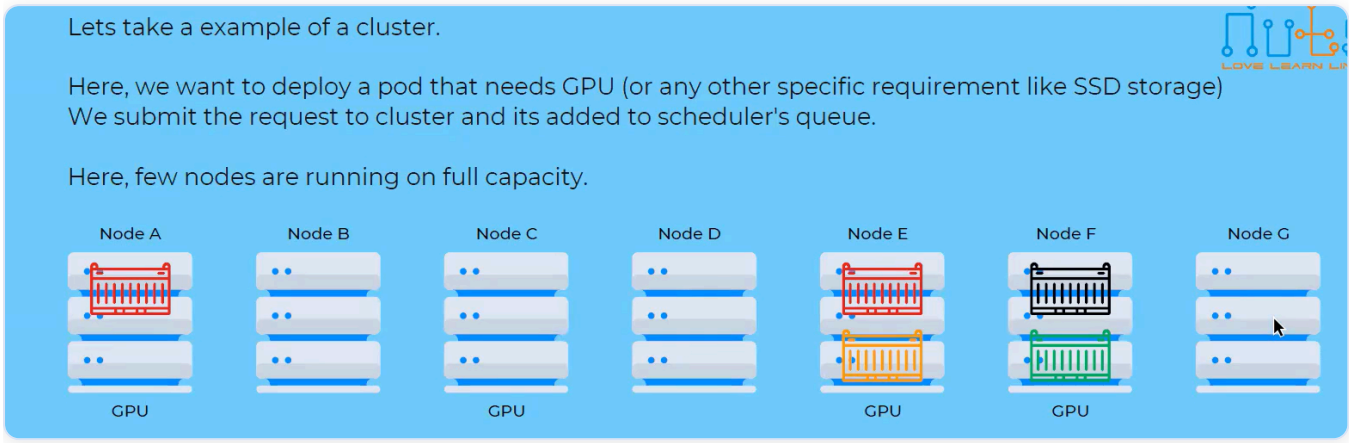
Step 2 — the Scheduler Processes Each Pending Pod in 2 Phases

1. **Scheduling Phase** — "what node should I choose?" Real decision-making happens here.
2. **Binding Phase** — "write to etcd that this pod belongs to that node." Updates the pod's entry: instead of `pending`, it now shows the real node name.

Inside the Scheduling Phase: Filtering + Scoring

1. **Filtering** (aka **predicates**) — remove nodes that flat-out *cannot* run this pod.
2. **Scoring** (aka **priorities**) — rank the *remaining* nodes best to worst, pick the winner.

Worked Example — Deploying a GPU-Requiring Pod



Cluster: Node A-G. Nodes A, C, E, F have GPUs. Node A is 50% GPU-utilized, Node C is 0% (fully free), Nodes E and F are 100% utilized.

#1 Filter phase: discard every node without a GPU — B, D, G eliminated immediately.

#2 Score phase: among A, C, E, F, score by *available* capacity. A: 50% available. C: 100% available. E, F: 0% available. **Node C wins**.

> **The general rule**
>
> Scoring favors the node with the most resources available *after* the new pod would run there — the emptiest viable node, not just "any node that fits."

The Default Filter Phase Has 13 Built-In Predicates

Common ones: `PodFitsHostPorts`, `PodFitsHost`, `PodFitsResources`, `PodMatchNodeSelector`, `NoVolumeZoneConflict`, `NoDiskConflict`, `MaxCSIVolumeCount`, `CheckNodeMemPressure`, `CheckNodePIDPressure`, `CheckNodeDiskPressure`, `CheckNodeCondition`, `PodToleratesNodeTaints`, `CheckVolumeBinding`.

We can further customize/influence this via: `nodeSelector`, `NodeAffinity`, `Pod Affinity`/`Anti-Affinity`, `Taints and Toleration`, `Topology Constraints`, `Scheduler Profiles` (available from v1.25+).

Node Selector/Affinity = Pod's Perspective; Taints/Tolerations = Node's Perspective

The single most important framing to remember

NodeSelector and (Node) Affinity are declared **on the Pod** — the Pod is choosing which node it wants. **Taints and Tolerations** are declared **on the Node** — the Node decides which pods it's willing to accept.

Node Selector — the Simplest Mechanism

2-step process: (1) label the node, (2) tell the pod to use that label via `nodeSelector`.

Node	CPU (key: cpu)	Disk (key: disk)
A	i7	sata
B	i5	ssd
C	xeon	ssd
D	amd	sata

Critical convention

The **key** must stay consistent across the whole cluster (always `cpu`, never `cpu` on one node and `processor` on another) — only the **value** varies per node.

```yaml
spec:
  template:
    spec:
      nodeSelector:
        cpu: i7
```

The critical limitation of nodeSelector

You cannot express multiple conditions — no AND, no OR. One key-value match only. If several nodes satisfy that one condition, the scheduler falls back to **scoring** to pick between them.

Managing Node Labels — the Actual Commands

```bash
kubectl describe node nodeone
# every node ships with some default labels already, e.g.:
# beta.kubernetes.io/arch=amd64, kubernetes.io/hostname=nodeone, ...

# Add labels
kubectl label node nodeone cpu=i7
kubectl label node nodeone mem=large
kubectl label node nodeone disk=sata
kubectl label node nodeone env=prod

# Change an EXISTING label's value - requires --overwrite or it's rejected
kubectl label node nodeone cpu=i7 --overwrite

# Remove a label entirely - trailing dash after the key, no value
kubectl label node nodeone cpu-

# See labels as columns across all nodes
kubectl get nodes -L cpu -L disk -L mem -L env
```

```bash
kubectl create -f mydep.yaml
kubectl get deploy
kubectl get rs
kubectl get pods -o wide      # shows NODE column directly

kubectl describe pod <pod-name> | grep -A2 "Node-Selectors"
# Node-Selectors:  disk=ssd
```

Editing a Running Deployment — What Happens to the ReplicaSet

```bash
kubectl edit deployment webapp
```

> **Key behavior**
>
> Editing a Deployment creates a **brand-new ReplicaSet** — but the **old ReplicaSet is scaled down to 0 replicas, not deleted**. The object itself persists.

```bash
kubectl get rs
# webapp-788497d9bd   2   2   0   2m43s   <- old RS, about to be scaled down
# webapp-cbf87bdf5    1   1   0   4s      <- new RS, rolling in
# ...moments later...
# webapp-788497d9bd   0   0   0   4m42s   <- scaled to 0, object still there
# webapp-cbf87bdf5    2   2   2   31s     <- fully rolled in and healthy
```

This is a deliberate Kubernetes design choice, not a bug:

1. **Rollback history management** — old ReplicaSets are kept so `kubectl rollout undo` has something to roll back to.
2. **Rollout flow control** — manages the transition smoothly (surge/unavailable settings) rather than an abrupt swap.

Scheduling by Hostname/IP — Bypassing the Scheduler Entirely

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nnappone
  namespace: learning
spec:
  nodeName: nodeone.example.com   # this will assign pod to specific node
  containers:
    - name: networknuts-app
      image: lovelearnlinux/webserver:v1
```

> **Important consequence**
>
> Setting `nodeName` directly **completely bypasses the scheduler** — no filtering, no scoring. `kubectl describe pod` on such a pod shows `Node-Selectors: <none>`, confirming no selector-based decision was ever made.

The Drawback of NodeSelector → Why Affinity/Anti-Affinity Exist

NodeSelector's core limitation (no AND/OR, single condition only) is exactly what **Affinity** and **Anti-Affinity** solve — real AND/OR/NOT-style logic via `matchExpressions` operators (`In` , `NotIn` , etc.).

- **Affinity** = "you like/love something" — attract toward a node or pod matching a condition.
- **Anti-Affinity** = "you dislike/hate something" — repel away from a node or pod matching a condition.
- Both come in 2 flavors: toward **nodes** or toward **other pods**.

⚠ Critical Timing Rule: Checked ONLY at Scheduling Time

This applies to `nodeSelector` too, not just affinity.

Worked example

Node A: `disk=ssd`, Node B: `disk=sata`, Node C: `disk=ssd`. A pod's `nodeSelector: {disk: sata}` lands on **Node B**. Later, you relabel Node B to `disk=ssd`. **Nothing happens to the already-running pod** — it keeps running on Node B, completely unaffected. Only **newly-created** pods from that point would see the new label.

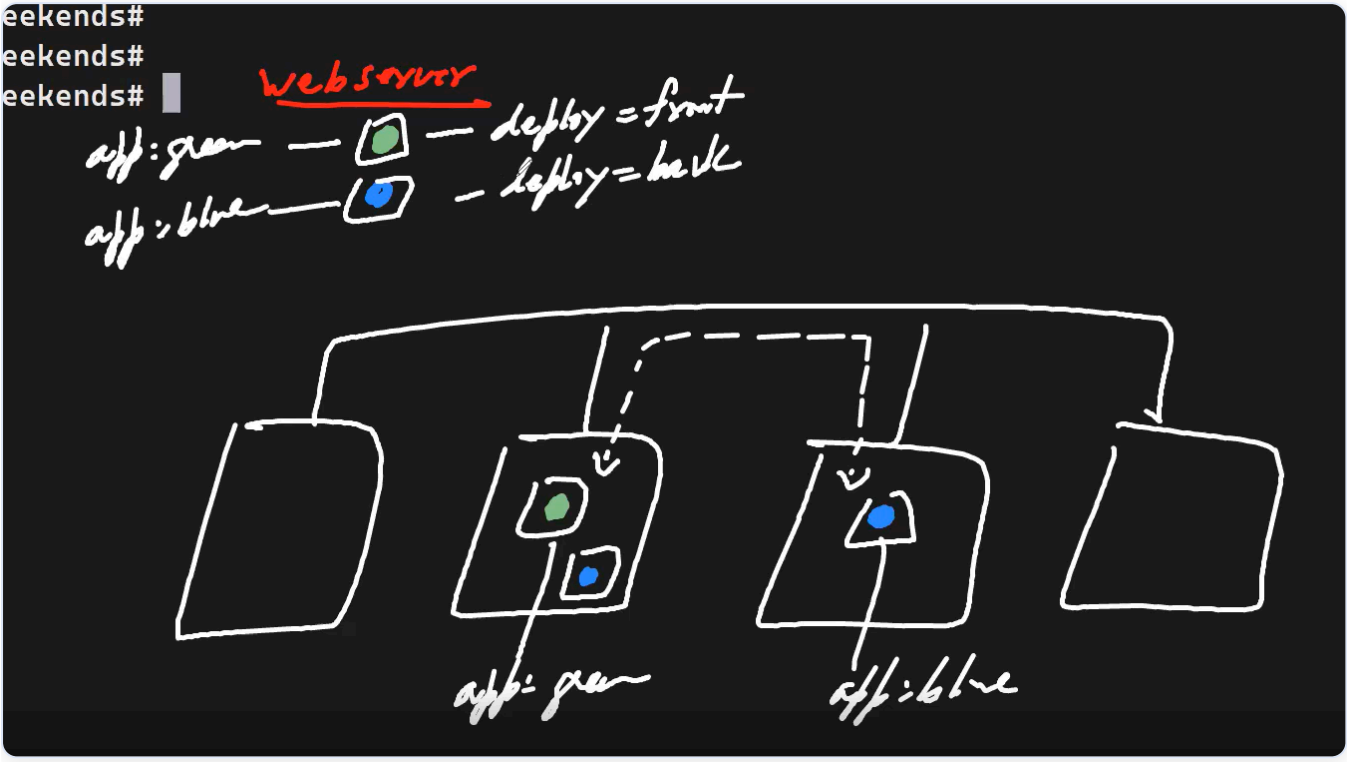
Node Affinity

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:    # HARD rule
      nodeSelectorTerms:
        - matchExpressions:
            - key: size
              operator: In
              values: [large, medium]
```

Rule type	Behavior
`requiredDuringSchedulingIgnoredDuringExecution`	**HARD** — if no node matches, the pod stays `Pending`.
`preferredDuringSchedulingIgnoredDuringExecution`	**SOFT** — tries to honor it, but falls back to scheduling anywhere valid if no match exists.

The `IgnoredDuringExecution` part of both names is the same timing rule made explicit: checked at scheduling, ignored (not re-checked) during the pod's lifetime.

Pod Affinity & Anti-Affinity



Why this exists: a `webserver` app has a `frontend` (`app: green`) and a `backend` (`app: blue`) deployment. Left alone, they could land on different nodes — fine, but adds network latency between tightly-coupled components. If they land on the **same** node, they benefit from shared local resources.

```yaml
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: app
              operator: In
              values: [nnappone]
        topologyKey: "kubernetes.io/hostname"
```

`topologyKey: kubernetes.io/hostname` is what actually means "same node."

Pod Anti-Affinity is the mirror case: don't run this pod on any node already running pods matching this label — e.g. keeping `green` and `blue` apart for fault isolation, so one node failure can't take out both tiers at once.

What I Practiced

- [ ] Traced a full deployment request end-to-end through pending → queue → filter+score → bind.
- [ ] Worked a GPU-node filtering/scoring example by hand.
- [ ] Labeled/relabeled/removed labels on real nodes and confirmed via `kubectl get nodes -L <key>`.
- [ ] Deployed a pod with `nodeSelector` and confirmed the applied selector via `describe pod`.
- [ ] Edited a running Deployment and watched a new ReplicaSet roll in while the old one scaled to 0 but stayed as an object.
- [ ] Deployed a pod using `nodeName` directly and confirmed the scheduler was fully bypassed.
- [ ] Demonstrated the "checked only at scheduling time" rule by relabeling a node under an already-running pod.
- [ ] Wrote both required (hard) and preferred (soft) node affinity rules with `In` operator `matchExpressions`.
- [ ] Wrote a `podAffinity` rule with `topologyKey: kubernetes.io/hostname`, and reasoned through the mirror-image `podAntiAffinity` case.

Merged into the master reference notes: k8s-master-notes.md §6 (Scheduling & Autoscaling) — new "Scheduler Internals, Node Selector, Node/Pod Affinity" addendum.